

Product Requirement Document

Local AI Software Team

1. Document Overview

Product Name

Local AI Software Team

Alternative names:

- SpecForge
- TeamForge AI
- ReqPilot
- DeliveryMind
- AgentPM

Product Type

Local-first developer productivity tool.

Main Interfaces

- CLI
- Local Web UI

Product Category

AI software delivery orchestrator.

Document Purpose

This PRD defines the product requirements for Local AI Software Team, a tool that helps developers turn rough software requirements into structured software delivery outputs, including clarified requirements, docs, tasks, test plans, implementation instructions, code patches, review reports, and traceability.

This document focuses on product behavior, user needs, core features, MVP scope, success criteria, and user workflows.

It does not focus on low-level technical implementation.

2. Product Summary

Local AI Software Team is a local-first AI software team that runs on the user's machine.

The product helps users turn vague software requirements into structured software work.

A user can enter a rough requirement such as:

“Thêm chức năng reset password bằng email OTP.”

The product will help generate:

- clarified requirement
- business rules
- assumptions
- open questions
- acceptance criteria
- technical design
- API design
- database design
- task breakdown
- test matrix
- implementation plan
- prompt for AI coding agents
- code patch if enabled
- test result
- review report
- final delivery report
- traceability matrix

The product does not replace existing AI coding tools. Instead, it organizes and coordinates tools such as Claude Code, Codex CLI, Gemini CLI, Aider, Cursor Agent CLI, and similar agents.

The product acts like a software team made of AI roles:

- Business Analyst
- Product Owner
- Project Manager
- Architect
- Developer
- QA Engineer
- Code Reviewer
- Security Reviewer
- Reporter

3. Product Vision

The long-term vision is:

A local AI software team that receives a rough requirement and helps move it toward reviewed, testable, traceable software delivery.

The product should make the user feel like they are managing a small software team, not just prompting one coding assistant.

Instead of this:

“AI, code this feature.”

The workflow becomes:

“AI BA, clarify this requirement. AI Architect, design it. AI PM, break it into tasks. AI Developer, implement it. AI QA, test it. AI Reviewer, review it. AI Reporter, summarize the delivery.”

4. Problem Statement

AI coding tools are becoming powerful, but they usually work best when the task is already clear.

Real software work often starts with vague input:

- “Add approval flow.”
- “Build admin dashboard.”
- “Export invoice CSV.”
- “Add booking cancellation.”
- “Create expense approval.”
- “Support refund processing.”

These requirements are often incomplete.

Common issues:

1. Business rules are missing.
2. Edge cases are unclear.
3. User roles are not defined.
4. Permission rules are not defined.
5. Acceptance criteria are missing.
6. API behavior is not specified.
7. Database impact is not analyzed.
8. Test cases are not prepared.
9. Developers manually convert business language into technical tasks.
10. AI coding agents may implement too early without enough context.

The real pain is not only code generation.

The real pain is:

Converting messy business intent into structured, testable, reviewable software delivery work.

5. Product Goals

Goal 1: Turn vague requirements into clear requirements

The product should help users transform rough input into:

- clarified requirement
- business rules
- user roles
- assumptions
- open questions
- acceptance criteria

Goal 2: Turn clear requirements into delivery plans

The product should generate:

- technical design
- API design
- database design
- task breakdown
- test matrix
- implementation plan

Goal 3: Coordinate existing AI coding tools

The product should use existing AI coding agents as workers instead of building a new AI model.

Supported tools may include:

- Claude Code
- Codex CLI
- Gemini CLI
- Aider
- Cursor Agent CLI
- OpenHands later

Goal 4: Keep everything local-first

The product should run locally by default.

User code, generated docs, logs, reports, and settings should remain on the user's machine unless the user explicitly enables external integrations.

Goal 5: Add trust through review and traceability

The product should help users understand:

- what requirement was implemented
- which task covers which requirement
- which files changed

- which tests verify the requirement
 - what passed
 - what failed
 - what still needs human review
-

6. Non-goals

The MVP should not try to solve everything.

The product should not initially include:

- desktop app
- required cloud backend
- user login
- billing system
- team SaaS dashboard
- full Slack bot event handling
- full Jira automation
- auto deployment
- auto merge
- enterprise admin console
- SSO
- centralized team management
- production monitoring integration

The product should also not promise:

“Any rough idea becomes a perfect production-ready project automatically.”

The realistic promise is:

“A rough requirement becomes structured docs, tasks, plans, prompts, and optionally code/test/review outputs with human control.”

7. Target Users

7.1 Primary Users

Solo Developer

Needs help turning ideas into structured implementation plans.

Pain points:

- does not have BA/PM/QA support
- spends time writing docs and tasks
- wants better prompts for coding agents
- wants safer code generation workflow

Backend Developer

Needs help analyzing enterprise requirements and generating technical delivery plans.

Pain points:

- requirements are vague
- business rules are complex
- needs API design, DB design, tests
- wants code generation but with control

Freelancer

Needs to convert client requests into professional delivery docs.

Pain points:

- clients give vague requirements
- needs estimate and task breakdown
- needs docs to look professional
- wants faster delivery

Small Team / Startup

Needs a lightweight AI software team without hiring full BA/QA/PM.

Pain points:

- lack of process
- lack of documentation
- no dedicated QA
- requirements change quickly

Vietnamese Software Team

Receives requirements in Vietnamese and needs structured technical outputs.

Pain points:

- Vietnamese business input must become technical English specs
- enterprise workflows are complex
- Jira tasks often lack clarity
- devs must manually ask many questions

8. Initial Niche

The best initial niche is:

Vietnamese requirement to Java/Spring Boot backend delivery workflow.

Reasons:

1. Vietnamese developers often receive vague Vietnamese requirements.
2. Enterprise backend requirements usually need clear business rules.
3. Java/Spring Boot is common in Vietnamese enterprise systems.
4. Backend features need API design, DB design, tests, and review.
5. The product can provide strong templates for Java/Spring Boot workflows.
6. This niche avoids directly competing with general AI coding tools.

Initial use cases should focus on:

- CRUD feature
- approval flow
- export report
- notification flow
- authentication feature
- business validation
- status transition workflow
- API + DB + test planning

9. Product Positioning

Bad Positioning

A cheaper Devin clone.

This is weak because Devin and similar products are broad and well-funded.

Better Positioning

A local AI software team above existing coding agents.

Strong Positioning

Local AI Software Team turns rough requirements into specs, tasks, test plans, code instructions, review reports, and traceability using your existing AI coding tools.

Short Pitch

From rough requirement to software delivery package.

Developer Pitch

AI BA + PM + Architect + QA layer for Claude Code, Codex CLI, Gemini CLI, and Aider.

Local-first Pitch

A local AI software team that keeps your code and project data on your machine.

10. Core User Scenarios

Scenario 1: Generate docs from a rough requirement

User enters:

“Thêm API export invoice CSV theo hotelId, date range, status.”

Product generates:

- clarified requirement
- business rules
- acceptance criteria
- API design
- task breakdown
- test matrix
- implementation prompt

Scenario 2: Prepare a Jira-ready task breakdown

User enters:

“Build expense approval system.”

Product generates:

- epic
- user stories
- subtasks
- acceptance criteria
- definition of done

Scenario 3: Use an AI coding agent safely

User enters a requirement and approves a plan.

Product generates an implementation prompt and sends it to a selected AI coding CLI.

Product then shows:

- changed files
- diff
- logs
- test result
- review report

Scenario 4: Review AI-generated work

Product reviews code changes and checks:

- whether requirement is covered

- whether tests exist
- whether unrelated files changed
- whether security risks exist
- whether final report is ready

Scenario 5: Generate traceability

Product maps:

- requirement
- acceptance criteria
- task
- code files
- test cases
- test results

This helps the user verify whether the work is actually done.

11. Main Product Workflow

11.1 Requirement Intake

User provides rough input.

Input sources for MVP:

- local web UI text box
- CLI command text
- pasted text from Slack/Jira/email/docs

Future input sources:

- uploaded document
- Jira ticket
- Slack thread
- GitHub issue
- meeting transcript

11.2 Requirement Clarification

The product generates:

- clarified requirement
- business goal
- user roles
- business rules
- main flow
- alternative flows
- edge cases
- assumptions

- open questions
- acceptance criteria

11.3 Scope Definition

The product defines:

- MVP scope
- out-of-scope items
- priority
- success criteria

11.4 Technical Planning

The product generates:

- technical design
- API design
- database design
- service flow
- affected modules
- risks
- implementation plan

11.5 Task Breakdown

The product generates:

- epic
- user stories
- subtasks
- dependency
- definition of done
- Jira-ready task format

11.6 Test Planning

The product generates:

- test matrix
- unit test scenarios
- integration test scenarios
- edge cases
- permission test cases
- acceptance criteria mapping

11.7 Coding Agent Preparation

The product generates:

- implementation prompt
- coding rules

- context summary
- files likely to change
- expected output

11.8 Optional Code Execution

If the user enables semi-auto mode, the product calls a selected AI coding CLI.

The product must require approval before code generation.

11.9 Testing

The product runs configured test/build commands if enabled.

It summarizes:

- pass/fail
- failed commands
- failed tests
- possible failure reasons

11.10 Review

The product reviews:

- diff
- changed files
- test result
- requirement coverage
- security risks
- missing tests
- unrelated changes

11.11 Final Report

The product generates:

- what was requested
 - what was planned
 - what was implemented
 - files changed
 - tests run
 - test result
 - review result
 - remaining risks
 - next steps
 - traceability matrix
-

12. Product Modes

12.1 Docs-only Mode

Purpose:

Create docs and plans without touching code.

Best for:

- MVP
- early planning
- requirement analysis
- users who want safe output only

Output:

- clarified requirement
- acceptance criteria
- technical design
- task breakdown
- test matrix
- final report

12.2 Assisted Mode

Purpose:

Generate high-quality implementation prompts for coding agents.

Best for:

- users who manually run Claude Code, Codex, Gemini CLI, or Aider
- users who want control

Output:

- all docs
- implementation prompt
- coding checklist
- test checklist

12.3 Semi-autonomous Mode

Purpose:

Let the product call one AI CLI after approval.

Best for:

- small features

- solo developers
- controlled implementation

Output:

- docs
- code patch
- changed files
- test result
- review report
- final report

12.4 Multi-agent Mode

Purpose:

Coordinate multiple AI roles and possibly different AI CLIs.

Best for:

- larger features
- advanced users
- future version

Example mapping:

- BA Agent: Gemini
 - Architect Agent: Claude
 - Developer Agent: Codex
 - Reviewer Agent: Claude
 - QA Agent: Codex
-

13. Core Features

13.1 Local Web UI

Description

A browser-based local UI running on localhost.

User Value

Makes the product easier to configure, review, and manage.

Requirements

The local web UI must support:

- dashboard

- settings
- new requirement
- run history
- run detail
- docs viewer
- prompt template viewer/editor
- logs viewer
- final report viewer

MVP Priority

Must-have.

13.2 CLI

Description

A command-line interface for running workflows.

User Value

Developers can use the tool directly in terminal.

Requirements

The CLI should support:

- init project
- start local web UI
- check environment
- create run
- generate docs
- generate plan
- generate report

MVP Priority

Must-have.

13.3 Requirement Clarifier

Description

Converts rough input into a clearer requirement.

User Value

Saves time and reduces misunderstanding.

Requirements

Should generate:

- clarified requirement
- business rules
- assumptions
- open questions
- user roles
- acceptance criteria

MVP Priority

Must-have.

13.4 Documentation Generator

Description

Generates structured product and technical documents.

User Value

Creates professional delivery artifacts quickly.

Requirements

Should generate:

- PRD summary
- SRS-style requirement
- technical design
- API design
- DB design
- test plan
- final report

MVP Priority

Must-have.

13.5 Task Breakdown Generator

Description

Turns requirement into implementation tasks.

User Value

Makes work easier to execute and track.

Requirements

Should generate:

- epic
- stories
- subtasks
- dependencies
- priority
- acceptance criteria
- definition of done

MVP Priority

Must-have.

13.6 Test Matrix Generator

Description

Maps requirement to test cases.

User Value

Improves reliability and reduces missing edge cases.

Requirements

Should generate:

- test case ID
- requirement mapping
- scenario
- test type
- expected result
- priority

MVP Priority

Must-have.

13.7 AI CLI Coordinator

Description

Coordinates existing AI coding tools.

User Value

Lets users use the tools they already pay for or already installed.

Requirements

Should support at least one coding agent in MVP+.

Possible agents:

- Claude Code
- Codex CLI
- Gemini CLI
- Aider

MVP Priority

Should-have for MVP+.

13.8 Code Diff Viewer

Description

Shows files changed by AI coding agent.

User Value

Allows user to review before accepting.

Requirements

Should show:

- changed files
- diff
- generated patch
- summary of changes

MVP Priority

Should-have for MVP+.

13.9 Test Runner Summary

Description

Runs configured tests and summarizes results.

User Value

Validates AI-generated code.

Requirements

Should show:

- command executed
- exit code
- pass/fail
- failed tests
- failure summary

MVP Priority

Should-have for MVP+.

13.10 Review Assistant

Description

Reviews generated output and code changes.

User Value

Adds quality control.

Requirements

Should check:

- requirement coverage
- code quality
- test coverage
- security risk
- unrelated changes
- missing edge cases

MVP Priority

Should-have for MVP+.

13.11 Traceability Matrix

Description

Maps requirement to tasks, code, and tests.

User Value

Shows whether the requirement is actually covered.

Requirements

Should map:

- requirement
- acceptance criteria
- task
- code files
- test cases
- test result
- status

MVP Priority

Should-have.

13.12 Settings

Description

Local configuration screen.

User Value

Allows customization without editing config files manually.

Requirements

Should allow user to configure:

- project name
- project path
- AI CLI tools
- agent role mapping
- test/build commands
- safety settings
- documentation settings
- integration tokens later

MVP Priority

Must-have.

14. Optional Integrations

Integrations are not required for the product to work.

14.1 GitHub

Purpose:

- create PR
- read CI result
- comment summary
- link delivery report

MVP status:

- not required
- can support GitHub CLI later

14.2 Jira

Purpose:

- create epic
- create story
- create subtask
- update task
- comment final report

MVP status:

- generate Jira-ready markdown first
- API integration later

14.3 Slack

Purpose:

- post progress updates
- post final report
- notify PR ready

MVP status:

- not required
- post-only integration later

14.4 Confluence / Notion

Purpose:

- publish docs
- publish technical design
- publish final report

MVP status:

- markdown export first
 - direct publishing later
-

15. Local-first Requirements

The product must work locally without cloud dependency.

Requirements

1. User must be able to run the product without creating an account.
 2. User must be able to use the product without backend cloud.
 3. Generated docs must be saved locally.
 4. Project config must be saved locally.
 5. Run history must be saved locally.
 6. Logs must be saved locally.
 7. External integrations must be optional.
 8. User must explicitly configure tokens for external integrations.
 9. Secrets must not be included in generated docs or logs.
 10. User code should not be uploaded to product-owned servers.
-

16. Safety Requirements

Because the product can call AI tools and possibly change code, safety is critical.

Must-have Safety Controls

1. User approval before code generation.
2. User approval before applying changes.
3. User approval before external updates.
4. Git diff review before acceptance.
5. Sensitive file protection.
6. Dangerous command blocking.
7. Token masking.
8. Local logs.
9. Clear assumptions and open questions.
10. Max iteration limit.

Sensitive Files

The product should protect files such as:

- `.env`
- `.env.*`
- `*.pem`
- `*.key`
- `credentials.json`
- production config files

Dangerous Actions

The product should not automatically:

- delete project files
 - push code
 - merge PR
 - deploy to production
 - modify production secrets
 - run unsafe shell commands
-

17. User Experience Requirements

17.1 Simplicity

The product should be usable by a developer in under 10 minutes.

A basic flow should be:

1. Install.
2. Run local UI.
3. Configure project.
4. Enter requirement.
5. Generate docs.
6. Review output.

17.2 Transparency

The product should always show:

- what step is running
- what agent is responsible
- what files are generated
- what assumptions were made
- what questions remain
- what changed
- what passed or failed

17.3 Control

The user should be able to:

- pause workflow
- cancel workflow
- regenerate one document
- edit prompt templates
- approve/reject coding
- rerun tests
- export reports

17.4 Reviewability

All outputs should be reviewable.

No important output should be hidden inside logs only.

18. Success Metrics

MVP Success Metrics

The MVP is successful if:

1. User can install and run the tool locally.
2. User can input a rough requirement.
3. Tool generates clear requirement docs.
4. Tool generates acceptance criteria.
5. Tool generates technical design.
6. Tool generates task breakdown.
7. Tool generates test matrix.
8. Tool saves run history locally.
9. User can view generated docs in local web UI.
10. User says output is better than using a generic AI chat prompt.

Product Quality Metrics

Track:

- number of successful runs
- number of regenerated docs
- number of user edits after generation
- number of accepted outputs
- average time from input to docs
- user rating for generated requirement
- user rating for generated task breakdown
- user rating for generated test matrix

Future Metrics

For coding workflow:

- percentage of code runs that produce diff
 - percentage of test runs that pass
 - percentage of review reports approved
 - number of traceability items covered
 - number of failed workflows
 - number of rollbacks
-

19. MVP Scope

MVP Name

Local AI Software Team MVP

MVP Goal

Help users convert rough requirements into structured software delivery documents.

MVP Must-have Features

1. CLI init.
2. Local web UI.
3. Settings page.
4. New requirement input.
5. Docs-only workflow.
6. Requirement clarification.
7. Acceptance criteria generation.
8. Technical design generation.
9. Task breakdown generation.
10. Test matrix generation.
11. Final report generation.
12. Run history.
13. Docs viewer.
14. Prompt templates.
15. Local file storage.

MVP Should-have Features

1. CLI doctor command.
2. Project type template.
3. Java/Spring Boot template.
4. Export markdown.
5. Basic traceability matrix.
6. Basic prompt editing.
7. Basic safety settings.

MVP Could-have Features

1. One AI CLI adapter.
2. Implementation prompt generator.
3. Diff viewer.
4. Test command runner.
5. Review report generator.

MVP Out of Scope

1. Cloud backend.
 2. User login.
 3. Billing.
 4. Desktop app.
 5. Full Jira automation.
 6. Full Slack bot.
 7. Auto deployment.
 8. Auto merge.
 9. Multi-user collaboration.
 10. Enterprise admin settings.
-

20. MVP User Flow

Flow: Generate delivery docs from rough requirement

1. User runs local web UI.
2. User opens New Requirement screen.
3. User enters rough requirement.
4. User selects Docs-only mode.
5. Product runs AI BA Agent.
6. Product generates clarified requirement.
7. Product runs AI Architect Agent.
8. Product generates technical design.
9. Product runs AI PM Agent.
10. Product generates task breakdown.
11. Product runs AI QA Agent.
12. Product generates test matrix.
13. Product runs Reporter Agent.
14. Product generates final report.
15. User reviews outputs in Run Detail screen.
16. User exports markdown files.

Expected output

- clarified-requirement.md
- acceptance-criteria.md
- technical-design.md
- task-breakdown.md
- test-matrix.md
- final-report.md

21. Example MVP Output

Input

“Thêm API export invoice CSV theo hotelId, date range, status.”

Clarified Requirement

The system should allow authorized users to export invoice data as a CSV file. The export should support filtering by hotel ID, date range, and invoice status.

Acceptance Criteria

1. User can export invoices by hotel ID.
2. User can filter invoices by fromDate and toDate.
3. User can filter invoices by status.
4. Export result must be in CSV format.
5. Empty result should return CSV with header only.
6. Invalid date range should return validation error.
7. Unauthorized user cannot export invoices.

API Design

GET /api/invoices/export

Query parameters:

- hotelId
- fromDate
- toDate
- status

Response:

- content type: text/csv
- file download

Task Breakdown

Epic:

- Invoice CSV Export

Stories:

1. Add export invoice API.
2. Add invoice query filters.
3. Generate CSV response.
4. Add validation and permission checks.
5. Add unit and integration tests.

Test Matrix

ID	Scenario	Expected Result
TC-001	Export with valid hotelId and date range	CSV file returned
TC-002	Export with invalid date range	Validation error
TC-003	Export with no matching invoices	CSV header only
TC-004	Unauthorized user exports invoice	Access denied

22. Future Scope

Phase 2: Assisted Coding

Features:

- implementation prompt generator
- coding agent instructions
- coding checklist
- expected changed files
- manual copy-to-agent workflow

Phase 3: Semi-auto Coding

Features:

- AI CLI adapter
- run selected coding CLI
- collect changed files
- generate diff
- run test commands
- generate review report

Phase 4: Traceability and Review

Features:

- requirement-to-test mapping
- requirement-to-code mapping
- review assistant
- security review
- risk report

Phase 5: External Integrations

Features:

- GitHub PR summary
- Jira task creation

- Slack progress update
- Confluence/Notion doc publishing

Phase 6: Team Workflow

Features:

- shared templates
 - policy presets
 - audit report
 - enterprise-friendly export
-

23. Product Risks

Risk 1: Users may prefer direct AI chat

Users may ask why they should use this instead of ChatGPT, Claude, Cursor, or Codex directly.

Mitigation:

- provide structured workflow
- provide reusable templates
- provide run history
- provide traceability
- provide safer code workflow
- produce better docs than one-off chat prompts

Risk 2: Scope becomes too large

The product idea can easily grow into Devin, Jira, Slack, GitHub, CI, docs, and deployment all at once.

Mitigation:

- start with docs-only workflow
- avoid full automation in MVP
- focus on requirement-to-delivery package
- focus on Java/Spring Boot first

Risk 3: AI output quality may be inconsistent

Generated docs may vary in quality.

Mitigation:

- use strict templates
- use role-based prompts
- allow user editing
- allow regeneration
- add quality checklist

Risk 4: Auto-code may be unsafe

AI coding agents may modify wrong files or produce wrong code.

Mitigation:

- approval gates
- diff review
- test runner
- protected files
- rollback
- local-only mode

Risk 5: Integrations may delay MVP

Jira, Slack, GitHub, Confluence, and Notion can take time.

Mitigation:

- start with markdown export
- create Jira-ready text instead of direct Jira API
- use GitHub CLI later
- make integrations optional

24. Assumptions

1. Target users are developers or technical users.
2. Users are comfortable running CLI tools.
3. Users may already use AI coding tools.
4. Users value local-first privacy.
5. Users need better requirement clarification.
6. Users need docs and task breakdown before code.
7. Java/Spring Boot is a good initial niche.
8. Full automation is less important than trustworthy workflow in early versions.

25. Open Questions

1. Should the first MVP support only docs-only mode?
 2. Should the first AI CLI adapter be Claude Code or Codex CLI?
 3. Should the initial template focus only on Java/Spring Boot?
 4. Should the local web UI allow editing generated docs?
 5. Should prompt templates be editable from MVP?
 6. Should traceability be included in MVP or MVP+?
 7. Should Jira integration start as export-only instead of API integration?
 8. Should the product support Vietnamese-only, English-only, or bilingual output?
 9. Should final reports be generated in Vietnamese, English, or both?
 10. Should the product name be broad or requirement-focused?
-

26. Release Plan

Alpha

Audience:

- creator only
- small personal projects
- test with Java/Spring Boot examples

Features:

- CLI
- local web UI
- docs-only workflow
- prompt templates
- markdown output

Private Beta

Audience:

- 5–10 developers
- Java/Spring Boot users
- Vietnamese developers

Features:

- better UI
- run history
- Java/Spring Boot template
- traceability matrix
- implementation prompt generator

Public Beta

Audience:

- broader developer community

Features:

- one AI CLI adapter
- test runner
- review report
- GitHub PR summary optional

Version 1.0

Audience:

- individual developers and small teams

Features:

- stable local workflow
 - docs generation
 - task generation
 - AI CLI coordination
 - test/review/report
 - optional Jira/GitHub/Slack integration
-

27. Pricing Consideration

The product is local-first and BYO AI tools, so the product owner does not pay for model usage.

Free

Features:

- local docs-only workflow
- basic templates
- markdown export
- limited run history

Pro

Features:

- advanced templates
- local AI CLI coordination
- traceability matrix
- review reports
- test runner
- more project templates

Team

Features:

- shared templates
- Jira/GitHub/Slack integrations
- team policy export/import
- audit reports
- advanced safety settings

Enterprise Later

Features:

- self-hosted options
- enterprise security
- centralized policy

- SSO if backend is added later
 - custom templates
 - internal model support
-

28. Acceptance Criteria for MVP

The MVP is acceptable when:

1. User can install and run the CLI.
 2. User can start local web UI.
 3. User can create a new requirement.
 4. Product can generate clarified requirement.
 5. Product can generate acceptance criteria.
 6. Product can generate technical design.
 7. Product can generate task breakdown.
 8. Product can generate test matrix.
 9. Product can generate final report.
 10. Product saves outputs locally.
 11. User can view run history.
 12. User can view generated docs in local web UI.
 13. User can export or copy generated docs.
 14. User can edit basic settings.
 15. Product works without login or backend cloud.
-

29. Final Product Requirement Summary

Local AI Software Team should start as a simple but powerful local-first tool.

The first version should focus on:

rough requirement → clear docs → task breakdown → test matrix → final report

The next version should add:

implementation prompt → AI CLI code generation → diff → test → review → traceability

The product should avoid becoming too broad too early.

The strongest wedge is:

Vietnamese/English vague requirements to structured software delivery artifacts for Java/
Spring Boot developers.

The product is not just another coding agent.

It is a local AI software team workflow that helps developers understand, plan, implement, verify, and report software work with more control and traceability.